

Task Manager Project — Study Notes

Java + Spring Boot + SQL | Triron James | September 2026

1. The 3-Layer Architecture

Every backend app is built in 3 layers. Each layer is a Java class with one job:

Layer	Class	Job
Model	Task.java	Represents data — maps Java object to SQL table row
Repository	TaskRepository.java	Talks to the database — runs SQL queries
Controller	TaskController.java	Handles web requests — connects browser to database

Rule: Each layer only talks to the one next to it. Controller → Repository → Database.

2. SQL Basics — CREATE TABLE

A SQL table is like an Excel spreadsheet. Columns are categories, rows are data.

```
CREATE TABLE tasks (  
  id    INT PRIMARY KEY,  
  title VARCHAR(255),  
  done  BOOLEAN  
);
```

SQL Code	Plain English
CREATE TABLE tasks	Make a new table named tasks
id INT PRIMARY KEY	A whole number column — unique for every row
title VARCHAR(255)	A text column up to 255 characters long
done BOOLEAN	A column that is either true or false

3. OOP ↔ SQL — The Key Connection

Java class fields map directly to SQL table columns. JPA translates between them automatically.

Java Field	SQL Column	What it stores
int id	id INT PRIMARY KEY	Unique number for each task
String title	title VARCHAR(255)	The task description text
boolean done	done BOOLEAN	Whether the task is complete

ORM = Object Relational Mapping. JPA is the tool that does this translation for you.

4. Task.java — The Model Class (Java Breakdown)

```
@Entity  
public class Task {  
  @Id
```

```

    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String title;
    private boolean done;
}

```

Java Code	What it means
@Entity	Links this Java class to a SQL table — the ORM bridge
@Id	This field is the Primary Key of the table
@GeneratedValue(...IDENTITY)	Auto-increment the id: 1, 2, 3... automatically
private int id	Whole number field — only accessible through getters/setters
private String title	Text field for the task name
private boolean done	True/false field for completion status

5. Getters and Setters — Encapsulation

Fields are private so nobody can change them directly. Getters and setters are the controlled doorways.

Method	Type	What it does
getId()	Getter	Returns the id value
getTitle()	Getter	Returns the title value
setTitle(String title)	Setter	Sets/changes the title value
isDone()	Getter	Returns true or false (boolean getters use 'is' not 'get')
setDone(boolean done)	Setter	Sets/changes the done value

Why private? Encapsulation — like a bank account. You can't change the balance directly. You go through deposit() and withdraw().

6. TaskRepository.java — The SQL Layer (Java Breakdown)

```

public interface TaskRepository extends JpaRepository<Task, Integer> {
}

```

Java Code	What it means
interface	A contract — defines what methods exist, Spring Boot fills them in
extends JpaRepository	Inherit all database methods from Spring Boot's JPA class
<Task, Integer>	We manage Task objects whose ID is an Integer

SQL Breakdown — Methods you get for FREE:

Java Method	SQL Generated Automatically
findAll()	SELECT * FROM tasks
findById(1)	SELECT * FROM tasks WHERE id = 1

save(task)	INSERT INTO tasks (title, done) VALUES (...)
deleteById(1)	DELETE FROM tasks WHERE id = 1

7. TaskController.java — The Web Layer (Java Breakdown)

```
@RestController
@RequestMapping("/tasks")
public class TaskController {

    private final TaskRepository taskRepository;

    public TaskController(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }

    @GetMapping
    public List<Task> getAllTasks() {
        return taskRepository.findAll();
    }

    @PostMapping
    public Task createTask(@RequestBody Task task) {
        return taskRepository.save(task);
    }
}
```

Java Code	What it means
@RestController	This class handles web requests and returns data
@RequestMapping('/tasks')	Every method in this class lives at the /tasks URL
private final TaskRepository	This controller needs a repository to access the database
public TaskController(...)	Constructor — runs once when Spring Boot creates this class
this.taskRepository = ...	Save the repository so all methods can use it
@GetMapping	Runs when browser visits GET /tasks
public List<Task> getAllTasks()	Returns a list of all Task objects
@PostMapping	Runs when someone sends data TO POST /tasks
@RequestBody Task task	Convert incoming JSON into a Task Java object
taskRepository.save(task)	Save the task to the database

SQL Breakdown — What runs when each URL is visited:

URL Visited	Java Method	SQL That Runs
GET /tasks	getAllTasks()	SELECT * FROM tasks
POST /tasks	createTask(task)	INSERT INTO tasks (title, done) VALUES (...)

8. The Full Request Flow

Here is exactly what happens when you visit localhost:8080/tasks:

Step	What happens
1	Browser sends GET request to localhost:8080/tasks
2	Spring Boot routes it to getAllTasks() in TaskController
3	Controller calls taskRepository.findAll()
4	Repository runs: SELECT * FROM tasks
5	H2 database returns all rows
6	JPA converts each row into a Task Java object
7	Controller returns the List of Task objects as JSON
8	Browser displays: [{"id":1,"title":"My first task","done":false}]

9. Key Terms Glossary

Term	Definition
OOP	Object Oriented Programming — organizing code into classes and objects
Class	A blueprint for creating objects (e.g. Task)
Object	One instance of a class (e.g. one specific task)
Field	A variable inside a class (e.g. String title)
Method	A function inside a class (e.g. getTitle())
Constructor	Special method that runs when an object is created
Encapsulation	Making fields private and using getters/setters to control access
Interface	A contract that defines method signatures without implementing them
Annotation	A label starting with @ that gives a class or method special powers
SQL	Structured Query Language — the language databases understand
JPA	Java Persistence API — translates Java objects to SQL rows
ORM	Object Relational Mapping — the process JPA uses to translate
REST API	A web service that responds to HTTP requests with data
JSON	JavaScript Object Notation — the data format APIs return
GET	HTTP request type for reading/fetching data
POST	HTTP request type for creating/sending new data
localhost:8080	Your own computer's address, door 8080
Maven	Java's dependency manager — like a shopping list for libraries
Spring Boot	Framework that turns Java into a web server